Preliminary Spec

Project Name: No Cutting!

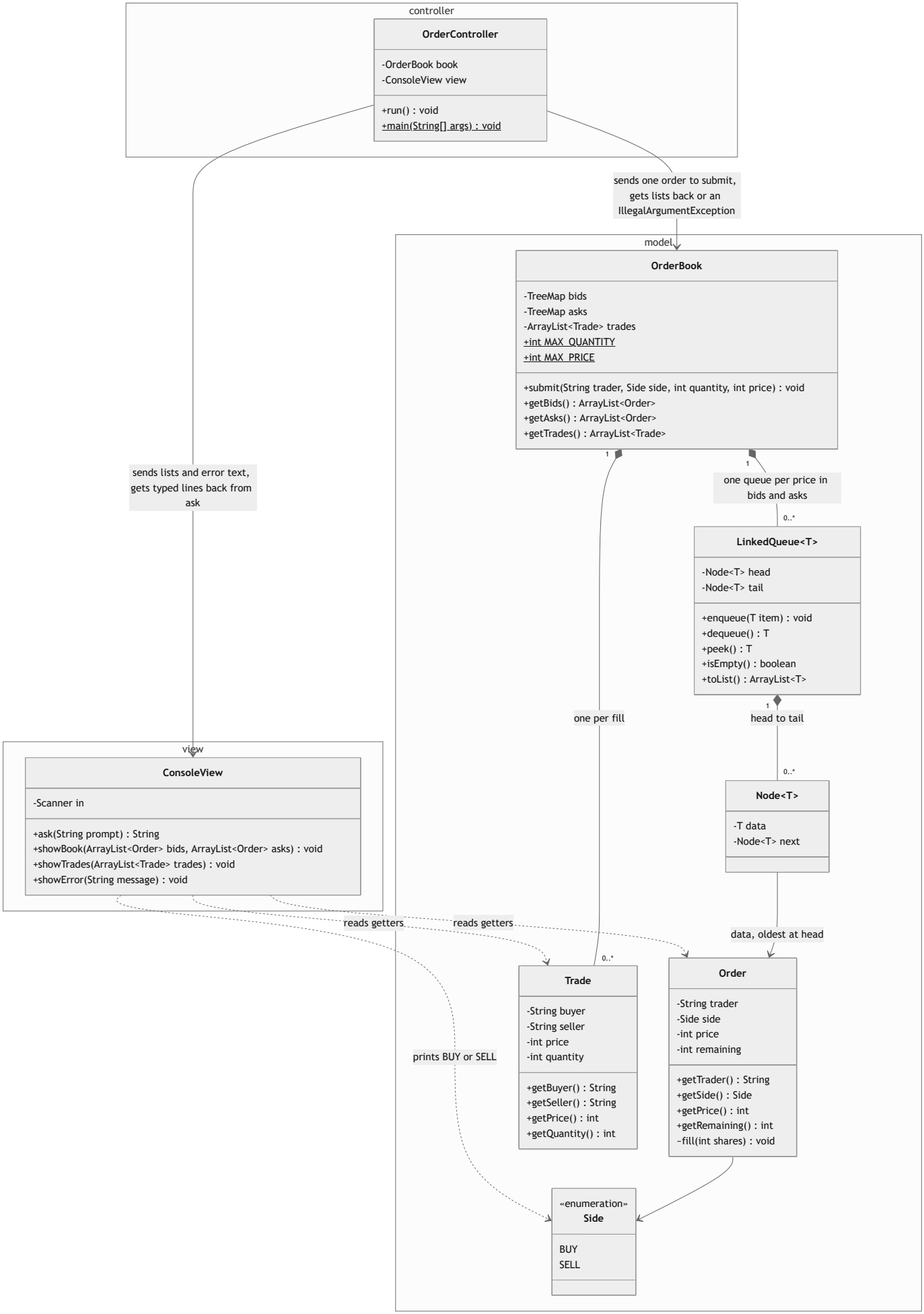
Class design, split by role

Class	Role	Responsibility
`LinkedQueue<T>`	Model	Holds items in arrival order; first in, first out.
`Side`	Model	Names the two sides of the book: BUY and SELL.
`Order`	Model	One trader's order: who, which side, what price, how many shares are still unfilled.
`Trade`	Model	Record of one fill: buyer, seller, price, shares.
`OrderBook`	Model	Keeps both sides of the book, checks and matches each new order, records the trades.
`ConsoleView`	View	Prints the book, the trade log and messages; reads what the user types.
`OrderController`	Controller	Turns typed text into a `submit` call and tells the view what to show.

`LinkedQueue` also has a private nested class `Node<T>` (model) that links one item to the next.

System diagram

Arrows run from the class that holds the reference to the class it uses, and each label says what moves along it. The only arrows that cross a package line start at `OrderController`, plus the dashed ones where `ConsoleView` reads `Order`, `Trade` and `Side`.


controller
OrderController
-OrderBook book
-ConsoleView view
+run() : void
+main(String[] args) : void
sends one order to submit, gets lists back or an IllegalArgumentException
model
OrderBook
-TreeMap bids
-TreeMap asks
-ArrayList<Trade> trades
+int MAX_QUANTITY
+int MAX_PRICE
+submit(String trader, Side side, int quantity, int price) : void
+getBids() : ArrayList<Order>
+getAsks() : ArrayList<Order>
+getTrades() : ArrayList<Trade>
1
1
one queue per price in bids and asks
0..*
LinkedQueue<T>
-Node<T> head
-Node<T> tail
+enqueue(T item) : void
+dequeue() : T
+peek() : T
+isEmpty() : boolean
+toList() : ArrayList<T>
1
head to tail
0..*
Node<T>
-T data
-Node<T> next
sends lists and error text, gets typed lines back from ask
view
ConsoleView
-Scanner in
+ask(String prompt) : String
+showBook(ArrayList<Order> bids, ArrayList<Order> asks) : void
+showTrades(ArrayList<Trade> trades) : void
+showError(String message) : void
one per fill
reads getters
reads getters
data, oldest at head
0..*
Trade
-String buyer
-String seller
-int price
-int quantity
+getBuyer() : String
+getSeller() : String
+getPrice() : int
+getQuantity() : int
Order
-String trader
-Side side
-int price
-int remaining
+getTrader() : String
+getSide() : Side
+getPrice() : int
+getRemaining() : int
~fill(int shares) : void
prints BUY or SELL
«enumeration»
Side
BUY
SELL

`bids` and `asks` are `TreeMap<Integer, LinkedQueue<Order>>`; the diagram shortens them to `TreeMap`. Filled diamonds mean the owner creates and holds the parts. `~` marks package-private, `$` marks static.

The model / view boundary

The view shows instances of `Order`, `Trade`, and `Side`. It also shows the trades and orderbook returned by the `OrderBook` methods `getTrades()`, `getBids()`, and `getAsks()`.

The view only reads `Order`, `Trade` and `Side` objects through their getters. It holds no reference to the `OrderBook`, so it has no access to the private TreeMaps or the `LinkedQueue`s inside them, and it cannot change an order.

The view is updated by `run` calling `showBook()` and `showTrades()` with the new lists of both. If a `submit()` call is rejected, it prints the error message only since the book and trades did not change.

This mechanism fits the system because only `run` calls `submit` and only `submit` can change the order book, so `run` always knows when the book has changed and can redraw the view right then.

Data & state

Where the Queue lives: inside `OrderBook`, as the values of the two TreeMaps. There is one `LinkedQueue<Order>` for each price that has at least one waiting order. It holds the `Order` objects waiting at that price, oldest at the head.

Class	Field	Type	Changes after construction?
`LinkedQueue<T>`	`head`	`Node<T>`	Yes, on every `dequeue` and on an `enqueue` into an empty queue
	`tail`	`Node<T>`	Yes, on every `enqueue`, and set to `null` when `dequeue` empties the queue
`Node<T>`	`data`	`T`	No
	`next`	`Node<T>`	Yes, when the next item is enqueued
`Order`	`trader`	`String`	No
	`side`	`Side`	No
	`price`	`int` (whole dollars)	No
	`remaining`	`int` (shares still unfilled)	Yes, goes down on each fill
`Trade`	`buyer`, `seller`	`String`	No
	`price`, `quantity`	`int`	No
`OrderBook`	`bids`	`TreeMap<Integer, LinkedQueue<Order>>`	The map is made once; its contents change on `submit`
	`asks`	`TreeMap<Integer, LinkedQueue<Order>>`	Same as `bids`
	`trades`	`ArrayList<Trade>`	The list is made once; it grows on each fill
	`MAX_QUANTITY`, `MAX_PRICE`	`static final int`, both 10,000	No
`ConsoleView`	`in`	`Scanner`	No
`OrderController`	`book`	`OrderBook`	No

Class	Field	Type	Changes after construction?
	`view`	`ConsoleView`	No

`Side` is an enum with two values and no fields. `bids` is sorted highest price first (`Collections.reverseOrder()`) and `asks` lowest first, so `firstKey()` is the best price on both sides.

Method signatures

Model: `LinkedQueue<T>`

Signature	Behavior	On bad input
`LinkedQueue()`	Makes an empty queue.	Takes no input.
`void enqueue(T item)`	Adds `item` at the tail.	`null`: throws `IllegalArgumentException`; queue unchanged.
`T dequeue()`	Removes and returns the head item.	Empty queue: throws `NoSuchElementException`.
`T peek()`	Returns the head item and leaves it in place.	Empty queue: throws `NoSuchElementException`.
`boolean isEmpty()`	Returns true when the queue holds no items.	Takes no input.
`ArrayList<T> toList()`	Returns a new list of the items, head first; the queue is unchanged.	Takes no input; an empty queue gives an empty list.

Model: `OrderBook`, `Order`, `Trade`

Signature	Behavior	On bad input
`OrderBook()`	Makes a book with no orders and no trades.	Takes no input.
`void submit(String trader, Side side, int quantity, int price)`	Fills the order against the other side, best price first and oldest first within a price; the unfilled remainder waits in line.	Throws `IllegalArgumentException` with a message naming the problem; the book and trade log are unchanged.
`ArrayList<Order> getBids()`	Returns a new list of waiting buy orders, highest price first, oldest first within a price.	Takes no input; an empty side gives an empty list.
`ArrayList<Order> getAsks()`	Returns a new list of waiting sell orders, lowest price first, oldest first within a price.	Takes no input; an empty side gives an empty list.
`ArrayList<Trade> getTrades()`	Returns a new list of every trade so far, oldest first.	Takes no input; no trades gives an empty list.
`Order`: `String getTrader()`, `Side getSide()`, `int getPrice()`, `int getRemaining()`	Each returns that field.	Takes no input.
`Trade`: `String getBuyer()`, `String getSeller()`, `int getPrice()`, `int getQuantity()`	Each returns that field.	Takes no input.

Only `OrderBook` makes `Order` and `Trade` objects, so their constructors and `Order.fill(int shares)` are package-private and receive values `submit` has already checked.

How `submit` works:

1. Check the input (see Where validation lives). If a check fails, throw before touching the book.

2. Find the best price on the other side: the lowest ask for a BUY, the highest bid for a SELL. It matches when the ask is at or below the buyer's price, or the bid is at or above the seller's price.
3. While the new order has shares left and the best price matches: `peek` at the head of that price's queue, which holds the oldest order, trade the smaller of the two share counts at the waiting order's price, add a `Trade` to `trades`, and lower `remaining` on both orders. `dequeue` the waiting order when it reaches 0, and remove the price from the TreeMap when its queue is empty.
4. If the new order still has shares left, `enqueue` it at its own price on its own side, making a new queue if the price is new.

Controller: `OrderController`

Signature	Behavior	On bad input
`OrderController(OrderBook book, ConsoleView view)`	Stores the book and the view it connects.	No check: `main` calls it once, passing the one `OrderBook` and `ConsoleView` it just made, so neither can be `null`.
`void run()`	Repeats: asks for trader, side, quantity and price, calls `submit`, shows the book and the trade log. Ends when the user types `quit` at the trader prompt.	Calls `showError` with a message and asks for the order again; the book is unchanged.
`static void main(String[] args)`	Makes one `OrderBook`, one `ConsoleView` and one `OrderController`, then calls `run`.	Ignores `args`.

View: `ConsoleView`

Signature	Behavior	On bad input
`ConsoleView()`	Opens a `Scanner` on `System.in`.	Takes no input.
`String ask(String prompt)`	Prints `prompt` and returns the line the user types.	Returns any text unchanged; `run` checks it.
`void showBook(ArrayList<Order> bids, ArrayList<Order> asks)`	Prints both waiting lines, one order per row.	No check: `run` passes the lists from `getBids` and `getAsks`, which are never `null`. An empty list prints an empty side.
`void showTrades(ArrayList<Trade> trades)`	Prints the trade log.	No check, for the same reason; an empty list prints an empty log.
`void showError(String message)`	Prints the message.	No check: every message comes from `run`.

Where validation lives

A rejected input never changes the book. For every row below the user sees a message and is asked for the order again.

Bad input from our scope	Caught by	What happens	Why there
Quantity or price of 0	`OrderBook.submit`	Throws `IllegalArgumentException`; `run` catches it and calls `showError`.	`submit` is the only way into the book, so one method guards it for the console, a GUI and JUnit.
Negative quantity or price	`OrderBook.submit`	Same.	Same.
Quantity or price above the max (10,000)	`OrderBook.submit`	Same.	Same. A number too big for an `int` is caught earlier, as text.
Blank trader name	`OrderBook.submit`	Same.	Same.
Self-trade: the trader already has a waiting order on the other side at a price the new order matches	`OrderBook.submit`	Same. Checked before any matching starts.	Only the book knows which orders are waiting.
Text ("ten") for quantity or price	`OrderController.run`, when `Integer.parseInt` fails	`run` calls `showError`; `submit` is never called.	`submit` takes an `int`, so text has to become a number before the model is called. Reading typed text is the controller's job.
A side other than buy/sell	`OrderController.run`, when `Side.valueOf(text.trim().toUpperCase())` fails	`run` calls `showError`; `submit` is never called.	`Side` is an enum, so the model cannot be handed a third side.
Cancelling a filled or nonexistent order ID (stretch)	`OrderBook.cancel(int orderId)`, added only if we build cancellation	Throws `IllegalArgumentException`; `run` catches it and calls `showError`.	Only the book knows which IDs are still waiting. A filled order has left its queue, so both cases look the same.

Test plan

`LinkedQueueTest` and `OrderBookTest` run in JUnit with no view or controller: they create only model objects and check return values and exceptions. Methods that take no input get an edge case, such as an empty queue

or book, in place of a bad-input case.

Method	Normal case and expected result	Bad-input case and expected result
`enqueue`	Enqueue "a" then "b". `toList()` is [a, b]; `isEmpty()` is false.	`enqueue(null)` throws `IllegalArgumentException`; `toList()` is unchanged.
`dequeue`	After enqueueing "a" then "b", `dequeue()` returns "a", then "b"; `isEmpty()` is true.	On an empty queue, throws `NoSuchElementException`.
`peek`	After enqueueing "a" then "b", `peek()` returns "a" twice; `toList()` is still [a, b].	On an empty queue, throws `NoSuchElementException`.
`isEmpty`	True on a new queue; false after one `enqueue`.	After one `enqueue` and one `dequeue`, true again.
`toList`	After "a", "b", "c", returns [a, b, c]; `dequeue()` still returns "a".	On an empty queue, returns an empty list.
`submit` (order waits)	On an empty book, alice SELL 100 at 25. `getAsks()` has one order: alice, 100 left, price 25. `getTrades()` is empty.	Each of quantity 0, -5, 10,001; price 0, -5, 10,001; trader "" or " "; side `null` throws `IllegalArgumentException`. `getBids()`, `getAsks()` and `getTrades()` are the same as before the call.
`submit` (fill, remainder waits)	alice SELL 100 at 25, then bob BUY 150 at 30. One trade: buyer bob, seller alice, 100 shares at 25. `getAsks()` is empty. `getBids()` has bob, 50 left, price 30.	alice SELL 100 at 25, then alice BUY 10 at 25 throws `IllegalArgumentException`. `getAsks()` still has alice, 100 left; `getTrades()` is empty.
`submit` (price first, then oldest first)	alice SELL 50 at 26, carol SELL 50 at 25, dave SELL 50 at 25, then bob BUY 120 at 26. Trades in order: carol 50 at 25, dave 50 at 25, alice 20 at 26. `getAsks()` has alice, 30 left.	Covered by the two rows above.
`getBids`	alice BUY at 24, bob BUY at 26, carol BUY at 26. Returns bob, carol, alice.	On an empty book, returns an empty list. Clearing the returned list leaves the next `getBids()` unchanged.
`getAsks`	alice SELL at 27, bob SELL at 25, carol SELL at 25. Returns bob, carol, alice.	On an empty book, returns an empty list. Clearing the returned list leaves the next `getAsks()` unchanged.
`getTrades`	After the price-first case above, returns its three trades, oldest first.	On a new book, returns an empty list. Clearing the returned list leaves the next `getTrades()` unchanged.
`Order` and `Trade` getters	Checked by every `submit` case above that reads a trader, price or share count.	They take no input.

Division of work

Piece	Owner	Builds
Model	Marcos Salas	`LinkedQueue`, `Side`, `Order`, `Trade`, `OrderBook`, `LinkedQueueTest`, `OrderBookTest`
View	Riya Shah	`ConsoleView`
Controller	Raghul Jayaprakash	`OrderController`

How we integrate:

1. The signatures in this spec are the contract. Changing one needs all three of us to agree first.
2. We share one GitHub repo with one branch per piece: `model` for Marcos, `view` for Riya and `controller` for Raghul. Each owner works only on their branch and merges it into `main` once their code compiles and their

tests pass.

3. The view and controller owners can start right away: `ConsoleView` needs only `Order` and `Trade`, which the model owner pushes first as fields and getters.
4. First join: the controller owner, Raghul, merges all three pieces on Monday, November 9 (prospective date), as soon as `LinkedQueueTest` and the first `OrderBookTest` case pass. They run `main` and type in the three `submit` normal cases from the test plan by hand, checking the printed book against the expected results.

Revised Scope

What changed since the scope doc	Why	Came from	Cost or saving in the build
Prices are whole dollars, stored as an `int`. The scope did not specify this.	Using an `int` makes prices easy to store as `TreeMap` keys without rounding. Invalid inputs like "10.5" and "ten" can be rejected using the same validation check.	Group discussion	Saves decimal-format validation and rounding code, but users cannot enter cents.
Self-trades are blocked by rejecting a new order if it could match a waiting order from the same trader at a compatible price.	Checking before matching begins ensures that a rejected order never changes the order book.	Marcos's own reflection	Adds a scan of eligible opposite-side orders before matching starts.
`LinkedQueue` gets a fifth method, `toList()`, which returns a snapshot of the waiting orders without changing the queue.	The view needs to print waiting orders in the terminal, and the self-trade check needs to inspect them without removing anything from the queue.	Group discussion	Adds one method and its tests, but avoids modifying the queue when reading orders.
The maximum price and quantity are both 10,000. The scope only mentioned a set maximum.	These limits keep individual price × quantity calculations within the Java `int` range. Cumulative totals may still require a `long`.	GenAI (Claude)	Adds two constants and prevents overflow in individual trade-value calculations.
Trades execute at the waiting order's price. The scope did not specify which price to use.	The waiting order was placed first, so using its price is consistent with the FIFO matching rule.	Marcos's own reflection	Requires no additional data structures or significant extra code.

Work Attribution

Section	Team member(s)	Contribution (how)
Class design	Marcos Salas, Riya Shah, Raghul Jayaprakash	Drafted by Marcos, GenAI-assisted (Claude); Marcos checked it visually and against our scope document; Riya and Raghul checked it manually
System diagram	Marcos Salas, Riya Shah, Raghul Jayaprakash	Drafted by Marcos, GenAI-assisted (Claude); Marcos checked it visually and against our scope document; Riya and Raghul checked it manually
Model / view boundary	Marcos Salas, Riya Shah, Raghul Jayaprakash	Drafted by Marcos, GenAI-assisted (Claude); Marcos, Riya and Raghul checked it manually
Data & state	Marcos Salas, Riya Shah, Raghul Jayaprakash	Drafted by Marcos, GenAI-assisted (Claude); Marcos, Riya and Raghul checked it manually
Method signatures	Marcos Salas, Riya Shah, Raghul Jayaprakash	GenAI-assisted (Claude); Marcos, Riya and Raghul checked it manually
Where validation lives	Marcos Salas, Riya Shah, Raghul Jayaprakash	GenAI-assisted (Claude); Marcos, Riya and Raghul checked it manually
Test plan	Marcos Salas, Riya Shah, Raghul Jayaprakash	Drafted by Marcos, GenAI-assisted (Claude); Marcos, Riya and Raghul checked it manually
Division of work	Raghul Jayaprakash, Marcos Salas, Riya Shah	Drafted by Raghul and Marcos, GenAI-assisted (Claude); Raghul reviewed the task assignments and integration plan; Marcos, Riya and Raghul checked it manually

Section	Team member(s)	Contribution (how)
Revised scope	Raghul Jayaprakash, Marcos Salas, Riya Shah	GenAI-assisted (Claude); Raghul reviewed and revised it for accuracy and consistency; Marcos, Riya and Raghul checked it manually
Work Attribution and GenAI note	Marcos Salas, Riya Shah, Raghul Jayaprakash	GenAI-assisted (Claude); Marcos, Riya and Raghul checked it manually

GenAI note

We gave Claude our scope document and the Deliverable 2 rubric and asked it for a minimal preliminary draft of this spec. Claude made that preliminary draft, helped format the document, and wrote the Mermaid code for the system diagram. It made suggestions on judgement calls; the ones we kept are the 10,000 max in Revised scope, putting every value check in `OrderBook.submit`, having the controller tell the view when to redraw, and splitting the classes into three packages. It also fact-checked the document, for example pointing out that our first rewrite of the model/view boundary said the view could read `OrderBook` fields, which the diagram contradicts. After the draft, Marcos rewrote the model/view boundary paragraphs, rejected Claude's ASCII system diagram and had it redone as a UML diagram, cut various redundant and unnecessary sentences, and corrected misleading wording in the `OrderController` constructor's description. All three of us then checked every section by hand.